

CUI_logo.jpg

COMSATS University Islamabad

Abbottabad Campus

Department of Computer Science

AI-Based Network Pathfinding and Attack Simulation System

Midterm Project Report

Artificial Intelligence · CSC 262

Student Name Sanullah Jadoon

Registration No. FA24-BCS-130

Section BCS-4C

Instructor Ms. Zeenat Zulfiqar

Submission Date May 28, 2026

Contents

1	Introduction	2
2	Problem Formulation	2
2.1	Graph Model	2
2.2	Network Nodes	2
2.3	Edges and Weights	3
2.4	Network Diagram	4
3	Algorithm Implementations	4
3.1	Breadth-First Search	4
3.2	Depth-First Search	5
3.3	Uniform Cost Search	6
3.4	A* Search	7
3.4.1	Heuristic Function	7
3.5	Hill Climbing	8
3.6	Minimax Algorithm	9
3.7	Alpha-Beta Pruning	10
4	Results and Comparative Analysis	11
4.1	Comparison Table	11
4.2	Bar Chart	12
4.3	Discussion	12
5	Conclusion	13

1. Introduction

Computer networks are made up of many connected devices such as routers, switches, servers, and workstations. When one device in a network is compromised, an attacker can use it as an entry point and try to move through the network to reach a more valuable target, like a database server.

This project builds a simulation of that kind of scenario using AI search algorithms. The network is represented as a graph where each device is a node and each connection between devices is an edge with a weight. The weight represents how easy that connection is to exploit, so a lower weight means a weaker link. The attacker starts at one node and the goal is to reach the target node.

Seven search algorithms were implemented from scratch and tested on the same network. The results were then compared to understand how each algorithm performs in terms of the path it finds, the total cost, and how much of the network it had to explore.

The algorithms covered are BFS, DFS, UCS, A* Search, Hill Climbing, Minimax, and Alpha-Beta Pruning.

2. Problem Formulation

2.1 Graph Model

The network is modelled as an undirected weighted graph. In formal terms:

$$G = (V, E, W)$$

where V is the set of nodes (devices), E is the set of edges (connections), and W assigns a weight to each edge (vulnerability score).

The attacker starts at node 0 and the goal is to reach node 7. Edge weights are integers between 1 and 8. A lower weight means the connection is easier to exploit.

2.2 Network Nodes

The network has 10 nodes representing different types of devices found in a typical company network. These are listed in Table 1.

Table 1: Network Nodes

Node	Device	Role
0	Entry Workstation	Start node (attacker entry point)
1	Router	Routes traffic between network segments
2	Switch	Connects devices on the same network
3	Firewall	Filters incoming and outgoing traffic
4	Web Server	Serves public web content
5	Mail Server	Handles email services
6	Internal Network	Core internal LAN
7	Database Server	Goal node (target)
8	Admin Workstation	High-privilege management machine
9	Backup Server	Stores backup data

2.3 Edges and Weights

Table 2 lists all 16 connections in the network with their vulnerability scores and a brief reason for the assigned weight.

Table 2: Network Edges and Vulnerability Weights

From	To	Weight	Reason
0	1	2	Weak link from entry workstation to router
0	2	4	Less-used path to switch
1	2	1	Internal, low-security router-switch link
1	3	5	Router to firewall, some protection
1	4	7	Exposed link to public web server
2	4	3	Switch to web server
2	5	6	Switch to mail server
3	6	4	Firewall to internal network
3	8	5	Firewall to admin workstation
4	6	5	Web server to internal network
4	9	4	Web server to backup server
5	6	3	Mail server to internal network
6	7	8	High-security link to database
6	8	2	Easy path from internal network to admin WS
8	7	3	Admin workstation has direct access to database
9	7	6	Backup server to database

2.4 Network Diagram

Figure 1 shows the full network layout with all nodes and edge weights.

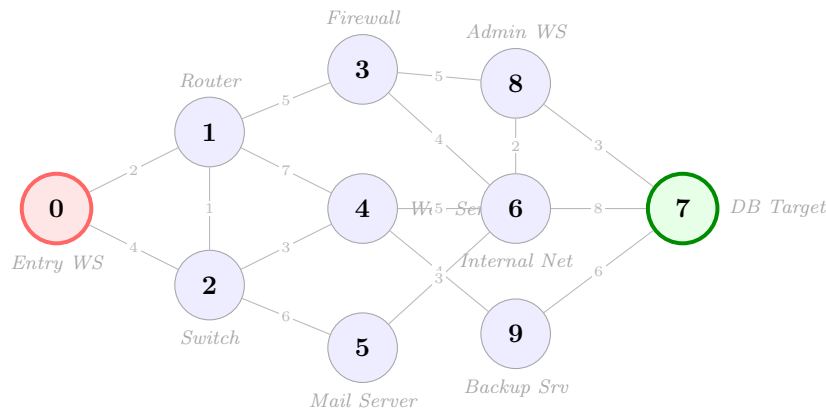


Figure 1: Network topology. Node 0 (red border) = attacker start. Node 7 (green border) = target database.

3. Algorithm Implementations

All seven algorithms were written from scratch in Python. No AI or graph libraries were used for the core logic. Each algorithm was run on the same network starting from node 0 and targeting node 7. The code for each algorithm is shown below along with the result it produced.

3.1 Breadth-First Search

BFS explores the network layer by layer. It visits all nodes one hop away before moving on to nodes two hops away. This is done using a queue (first-in, first-out).

The result is the path with the fewest steps to reach the target, but it ignores edge weights so the cost might not be the lowest.

Complete: Yes. **Optimal (cost):** No.

```

1 from collections import deque
2 import time
3
4 def bfs(graph, start, goal):
5     t = time.perf_counter()
6     queue = deque([(start, [start], 0)])
7     visited = set()
8     expanded = 0
9

```

```

10 while queue:
11     node, path, cost = queue.popleft()
12     if node in visited:
13         continue
14     visited.add(node)
15     expanded += 1
16
17     if node == goal:
18         ms = round((time.perf_counter() - t) * 1000, 4)
19         return path, cost, expanded, ms
20
21     # add unvisited neighbors to the queue
22     for neighbor, weight in graph[node]:
23         if neighbor not in visited:
24             queue.append((neighbor, path + [neighbor], cost +
25                           weight))
26
27 return [], -1, expanded, 0

```

Listing 1: Breadth-First Search

Result: $0 \rightarrow 1 \rightarrow 3 \rightarrow 6 \rightarrow 7$, cost = 19, nodes expanded = 10.

3.2 Depth-First Search

DFS goes as deep as possible along one branch before backtracking to try other options. It uses a stack (last-in, first-out).

It can find a path quickly but often picks a longer, more expensive one because it commits to the first direction without looking at alternatives.

Complete: Yes (with visited set). **Optimal:** No.

```

1 def dfs(graph, start, goal):
2     t = time.perf_counter()
3     stack = [(start, [start], 0)]
4     visited = set()
5     expanded = 0
6
7     while stack:
8         node, path, cost = stack.pop()
9         if node in visited:
10             continue
11         visited.add(node)
12         expanded += 1
13
14         if node == goal:
15             ms = round((time.perf_counter() - t) * 1000, 4)

```

```

16         return path, cost, expanded, ms
17
18         # reversed so left-to-right traversal order is maintained
19         for neighbor, weight in reversed(graph[node]):
20             if neighbor not in visited:
21                 stack.append((neighbor, path + [neighbor], cost +
22                             weight))
23
24     return [], -1, expanded, 0

```

Listing 2: Depth-First Search

Result: $0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 3 \rightarrow 8 \rightarrow 7$, cost = 23, nodes expanded = 8.

3.3 Uniform Cost Search

UCS always picks the node with the lowest accumulated cost from the start. It uses a min-heap (priority queue). This means when the goal is reached, the path to it is guaranteed to be the cheapest one possible.

Complete: Yes. **Optimal:** Yes — always finds minimum cost path.

```

1  import heapq
2
3  def ucs(graph, start, goal):
4      t = time.perf_counter()
5      heap = [(0, start, [start])]    # (cost, node, path)
6      visited = set()
7      expanded = 0
8
9      while heap:
10         cost, node, path = heapq.heappop(heap)
11         if node in visited:
12             continue
13         visited.add(node)
14         expanded += 1
15
16         if node == goal:
17             ms = round((time.perf_counter() - t) * 1000, 4)
18             return path, cost, expanded, ms
19
20         for neighbor, weight in graph[node]:
21             if neighbor not in visited:
22                 heapq.heappush(heap, (cost + weight, neighbor, path +
23                                     [neighbor]))
24
25     return [], -1, expanded, 0

```

Listing 3: Uniform Cost Search

Result: $0 \rightarrow 1 \rightarrow 3 \rightarrow 8 \rightarrow 7$, cost = **15** (optimal), nodes expanded = 10.

3.4 A* Search

A* combines UCS with a heuristic estimate. For each node it calculates:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual cost from the start to node n , and $h(n)$ is an estimate of how much cost remains to reach the goal. The algorithm expands the node with the lowest $f(n)$ value, which means it can skip parts of the network that are unlikely to lead to a good solution.

3.4.1 Heuristic Function

I designed $h(n)$ to represent the estimated remaining vulnerability cost from each node to the database. The values are based on how many security barriers separate a node from the goal and the minimum edge weights along that remaining path. The heuristic is admissible, meaning it never overestimates the true remaining cost, so A* will always find an optimal or near-optimal path.

Table 3: Heuristic values $h(n)$ for each node

Node	Device	$h(n)$	Reasoning
0	Entry Workstation	18	Far from goal, multiple hops
1	Router	14	One step closer
2	Switch	13	Similar position to router
3	Firewall	10	Past major security barrier
4	Web Server	9	Mid-way in the network
5	Mail Server	9	Similar position to web server
6	Internal Network	5	Close to target
7	Database (Goal)	0	This is the goal
8	Admin Workstation	3	One hop from target
9	Backup Server	7	Alternative route

```

1 HEURISTIC = {0: 18, 1: 14, 2: 13, 3: 10,
2             4: 9, 5: 9, 6: 5, 7: 0, 8: 3, 9: 7}
3
4 def astar(graph, start, goal, h):
5     t = time.perf_counter()
6     heap = [(h[start], 0, start, [start])]    # (f, g, node, path)

```

```

7     visited = set()
8     expanded = 0
9
10    while heap:
11        f, g, node, path = heapq.heappop(heap)
12        if node in visited:
13            continue
14        visited.add(node)
15        expanded += 1
16
17        if node == goal:
18            ms = round((time.perf_counter() - t) * 1000, 4)
19            return path, g, expanded, ms
20
21        for neighbor, weight in graph[node]:
22            if neighbor not in visited:
23                new_g = g + weight
24                new_f = new_g + h[neighbor]    # f = g + h
25                heapq.heappush(heap, (new_f, new_g, neighbor, path + [
26                    neighbor]))
27
28    return [], -1, expanded, 0

```

Listing 4: A* Search

Result: $0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow 7$, cost = 16, nodes expanded = **7** (fewest of all).

3.5 Hill Climbing

Hill Climbing is a local search method. At each node, it looks at all unvisited neighbours and moves to the one with the lowest heuristic value. It does not maintain a search tree and cannot go back to try a different path.

The main problem is that it can get stuck at a local maximum. This happens when every available neighbour has a heuristic value that is equal to or worse than the current node. Since the algorithm only looks one step ahead and cannot backtrack, it stops there even if the goal has not been reached yet.

Complete: No. **Optimal:** No.

```

1 def hill_climbing(graph, start, goal, h):
2     t = time.perf_counter()
3     current = start
4     path = [current]
5     cost = 0
6     expanded = 0

```

```

7     visited = {current}
8
9     while current != goal:
10         expanded += 1
11         neighbors = [(nb, w) for nb, w in graph[current] if nb not in
12                     visited]
13
14         if not neighbors:
15             ms = round((time.perf_counter() - t) * 1000, 4)
16             return path, cost, expanded, ms, "stuck: no moves left"
17
18         # pick the neighbor with the lowest heuristic value
19         best_nb, best_w = min(neighbors, key=lambda x: h[x[0]])
20
21         # local maximum check: best neighbor is no better than current
22         # node
23         if h[best_nb] >= h[current] and current != start:
24             ms = round((time.perf_counter() - t) * 1000, 4)
25             return path, cost, expanded, ms, f"stuck at node {current}"
26
27         visited.add(best_nb)
28         path.append(best_nb)
29         cost += best_w
30         current = best_nb
31
32     ms = round((time.perf_counter() - t) * 1000, 4)
33     return path, cost, expanded, ms, "goal reached"

```

Listing 5: Hill Climbing with Local Maximum Detection

Result: $0 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7$, cost = 20, nodes expanded = 4.

3.6 Minimax Algorithm

Minimax is used when two players compete against each other. In this simulation:

- The **Attacker (Maximiser)** tries to move towards the target, aiming for the highest heuristic score.
- The **Defender (Minimiser)** tries to block the attacker, aiming for the lowest heuristic score.

The algorithm explores the game tree to a depth of 5 and assumes both players always make the best possible move.

3.7 Alpha-Beta Pruning

Alpha-Beta Pruning is an optimisation added on top of Minimax. It tracks two values:

- α — the best score the attacker is guaranteed so far
- β — the best score the defender is guaranteed so far

If $\beta \leq \alpha$ at any point, the current branch cannot change the final result and is pruned (skipped). This reduces the number of nodes evaluated without affecting the answer.

```

1 node_count = 0
2
3 def minimax(graph, node, depth, is_max, goal, h,
4             path, visited, a, b, pruning):
5     global node_count
6     node_count += 1
7
8     if node == goal or depth == 0:
9         return h[node], path
10
11     neighbors = [(nb, w) for nb, w in graph[node] if nb not in visited
12                 ]
13     if not neighbors:
14         return h[node], path
15
16     best_path = path
17
18     if is_max:
19         best = float('-inf')
20         for nb, _ in neighbors:
21             val, p = minimax(graph, nb, depth - 1, False, goal, h,
22                             path + [nb], visited | {nb}, a, b,
23                             pruning)
24
25             if val > best:
26                 best, best_path = val, p
27             if pruning:
28                 a = max(a, best)
29                 if b <= a:
30                     break          # beta cutoff
31         return best, best_path
32     else:
33         best = float('inf')
34         for nb, _ in neighbors:
35             val, p = minimax(graph, nb, depth - 1, True, goal, h,
36                             path + [nb], visited | {nb}, a, b,
37                             pruning)
38
39             if val < best:
40                 best, best_path = val, p

```

```

36         if pruning:
37             b = min(b, best)
38             if b <= a:
39                 break          # alpha cutoff
40         return best, best_path

```

Listing 6: Minimax with Optional Alpha-Beta Pruning

Minimax result: $0 \rightarrow 1 \rightarrow 3 \rightarrow 6 \rightarrow 7$, cost = 19, nodes evaluated = 95.

Alpha-Beta result: Same path, same cost, nodes evaluated = 66 (30% fewer).

4. Results and Comparative Analysis

4.1 Comparison Table

Table 4 shows the output of all seven algorithms on the same network.

Table 4: Algorithm Comparison Results

Algorithm	Path	Cost	Nodes	Time (ms)
BFS	$0 \rightarrow 1 \rightarrow 3 \rightarrow 6 \rightarrow 7$	19	10	0.031
DFS	$0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 3 \rightarrow 8 \rightarrow 7$	23	8	0.011
UCS	$0 \rightarrow 1 \rightarrow 3 \rightarrow 8 \rightarrow 7$	15	10	0.027
A*	$0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow 7$	16	7	0.009
Hill Climbing	$0 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7$	20	4	0.011
Minimax	$0 \rightarrow 1 \rightarrow 3 \rightarrow 6 \rightarrow 7$	19	95	0.082
Alpha-Beta	$0 \rightarrow 1 \rightarrow 3 \rightarrow 6 \rightarrow 7$	19	66	0.051

4.2 Bar Chart

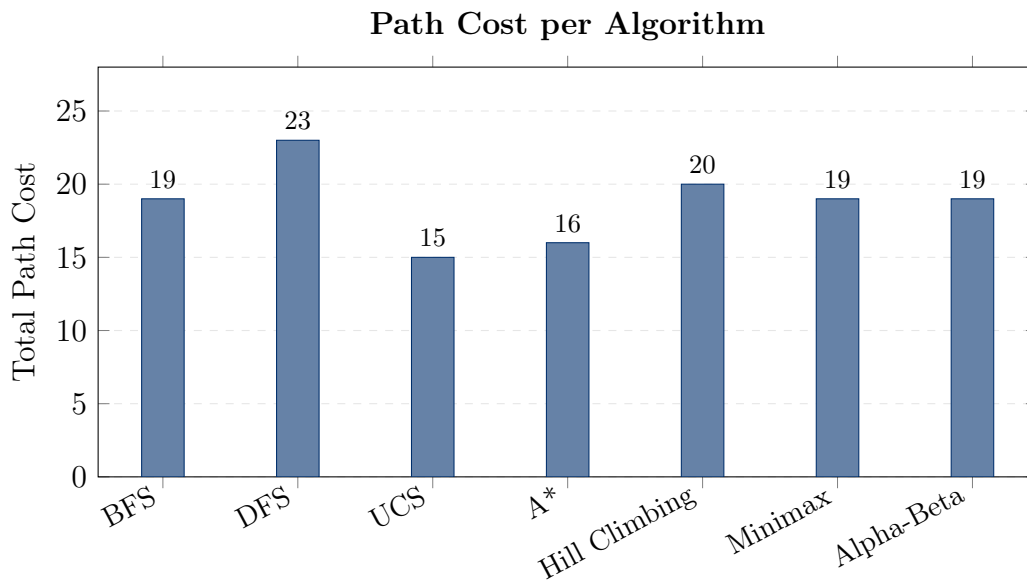


Figure 2: UCS produced the lowest cost path (15). DFS produced the highest cost path (23).

4.3 Discussion

Path cost. UCS found the cheapest path with a cost of 15. The path goes $0 \rightarrow 1 \rightarrow 3 \rightarrow 8 \rightarrow 7$, which passes through the Admin Workstation. This is a shorter route because the Admin Workstation has a direct, low-cost connection to the database. A* was close with cost 16, taking a slightly different route through the Internal Network.

Efficiency. A* expanded only 7 nodes, which is the lowest among all proper search algorithms. The heuristic guided it away from nodes that were unlikely to lead to an optimal solution.

DFS. Found the worst path (cost 23) because it followed one branch all the way through without checking if a cheaper alternative existed.

Hill Climbing. Expanded only 4 nodes, but this is not necessarily good. It simply followed the greedy heuristic and stopped as soon as the goal was reached. If the network had a different structure, it could have stopped at a local maximum and failed completely.

Minimax vs Alpha-Beta. Both produced the same result. Alpha-Beta evaluated 29 fewer nodes, which is a 30% saving. In a larger game tree this difference would be much more significant.

5. Conclusion

This project gave me a good understanding of how different search algorithms work and when each one is useful. By running all seven on the same network problem, I could directly see the differences in the paths they found, the costs, and how efficiently they searched.

The key findings are:

- UCS is the right choice when you need the lowest-cost path and have no additional information about the network.
- A* is more practical in real use because the heuristic helps it find a good path while exploring fewer nodes.
- BFS and DFS are simple to implement but should not be used when the cost of each connection matters.
- Hill Climbing is fast but risky. Its greedy nature makes it vulnerable to getting stuck at local maxima.
- Minimax with Alpha-Beta Pruning is well suited for simulating attack-versus-defence scenarios. The pruning optimisation is important for keeping computation manageable.

Some challenges I ran into: making sure BFS, DFS, and UCS handled cycles correctly using a visited set, designing a heuristic for A* that is admissible, and preventing the Minimax recursion from going into cycles on the graph.

If I were to extend this work, I would load the network from an external file so the graph can be changed without editing the code, and test the algorithms on larger networks to see how well they scale.

Sanullah Jadoon FA24-BCS-130 BCS-4C

Artificial Intelligence (CSC 262) — COMSATS University Islamabad, Abbottabad Campus

Submitted: May 28, 2026